

# Module 5: Control Flow - Decision Making

## 5.1 The if Statement

- The if statement allows the program to execute a block of code only when a condition is true
- Syntax

```
if (condition) { statements; }
```

- The condition inside parentheses must evaluate to true or false
- If the condition is true, the block inside braces executes
- If the condition is false, the block is skipped entirely
- Single-statement if without braces (works but is error-prone and not recommended)

```
if (x > 0) cout << "Positive";  
  
Multi-statement if with braces (always recommended)  
if (x > 0) {  
    cout << "Positive";  
    cout << " number";  
}
```

- Nested if: an if statement inside another if statement  
Used when a second condition only makes sense after the first is true

## 5.2 The if-else Statement

- Adds a block that executes when the condition is false
- Syntax

```
if (condition) {  
    // executes when condition is true  
} else {  
    // executes when condition is false  
}
```

- Example: checking if a number is even or odd

```
if (number % 2 == 0) {  
    cout << "Even";  
} else {  
    cout << "Odd";  
}
```

- Common mistake: putting a semicolon after the if condition
- `if (x > 0);` means do nothing if true, then always execute the next line

## 5.3 The else if Ladder

- Used when there are more than two possibilities to check
- Only the first matching block executes; the rest are skipped
- Syntax

```

if (condition1) {
// executes if condition1 is true
} else if (condition2) {
// executes if condition1 was false and condition2 is true
} else if (condition3) {
// executes if both above were false and condition3 is true
} else {
// executes if none of the above were true
}

```

- Example: letter grade from percentage

```

if (percent >= 90) cout << "A";
else if (percent >= 80) cout << "B";
else if (percent >= 70) cout << "C";
else if (percent >= 60) cout << "D";
else cout << "F";

```

- Difference between chained else if and multiple separate if statements
- In a chain, once one condition matches, the rest are skipped
- With separate if statements, all of them are checked every time

## 5.4 The switch Statement

- Used when comparing one variable against several possible fixed integer or character values

- More readable than a long else if ladder when comparing a single variable

- Syntax

```

switch (variable) {
case value1:
// code for value1
break;
case value2:
// code for value2
break;
default:
// code when no case matches
}

```

- How switch works
- The variable is evaluated once
- The program jumps to the matching case label
- Code executes from that point until a break is reached
- The break statement
- Exits the switch block immediately
- Without break, execution "falls through" to the next case automatically
- Fall-through is sometimes intentional but usually a bug
- The default case
- Optional, runs when no case label matches

- Similar to the else in an if-else chain
- Limitations of switch
- Only works with integer types: int, char, short, long, enum
- Does not work with float, double, or string
- Cannot check ranges (no case `x > 5:`)

## 5.5 Practical Decision-Making Examples

- Check if a number is positive, negative, or zero
- Find the largest of two numbers using if-else
- Find the largest of three numbers using nested if or else if
- Check if a year is a leap year (divisible by 4, except centuries unless divisible by 400)
- Simple calculator using switch for the operator (+, -, \*, /)
- Grade calculator: take percentage as input and output letter grade